

Lecture 06: ArrayList and OOP Practice

Introduction to Programming with Java

Today's learning goals

By the end of this lecture you should be able to:

1. Contrast fixed-size arrays with growable `ArrayList`
2. Add, get, remove, and loop over list elements
3. Store objects in an `ArrayList`
4. Combine classes + lists in a small end-to-end example

Agenda

1. Arrays vs ArrayList
2. Basic ArrayList operations
3. Lists of objects
4. Enhanced for-loop with lists
5. Mini case study: a simple roster
6. Practice ideas and pitfalls

Arrays are fixed length

- `new int[10]` — size chosen up front
- Cannot “grow” if you need an 11th element
- Resizing yourself means create larger array + copy

Many programs need lists that grow as users add items.

ArrayList in one slide

- Part of `java.util`
- Holds objects (for numbers use wrapper types: `Integer`, `Double`)
- Grows as you add
- Indexed access like arrays: `get(i)`

Import and create

```
1 import java.util.ArrayList;
2
3 ArrayList<String> names = new ArrayList<>();
4 names.add("Ada");
5 names.add("Alan");
6 names.add("Grace");
7 System.out.println(names.size()); // 3
8 System.out.println(names.get(0)); // Ada
```

`ArrayList<String>` means “list of String”. The `<>` form is the diamond operator (type inferred).

Generics (intuition only)

- `ArrayList<Student>` — only `Student` references (at compile time)
- Helps catch mistakes early
- You do not need the full generics theory today

Quiz 1 — Arrays vs ArrayList

Quiz

Which is a good reason to prefer ArrayList over a plain array?

1. Lists can never hold objects
2. The collection can grow as you add elements
3. Arrays cannot store `int` values
4. ArrayList replaces the need for classes

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. ArrayList grows with add; arrays have fixed length after creation.

add, get, set, remove, size

```
1 ArrayList<String> names = new ArrayList<>();  
2 names.add("Ada");  
3 names.add("Alan");  
4 names.set(1, "Alice");           // replace index 1  
5 String first = names.get(0);  
6 names.remove(0);                 // remove by index  
7 System.out.println(names.size());
```

Useful checks

```
1 if (names.isEmpty()) {  
2     System.out.println("No names yet");  
3 }  
4 if (names.contains("Ada")) {  
5     System.out.println("Found Ada");  
6 }  
7 int i = names.indexOf("Alan"); // -1 if missing
```

Numbers: wrapper types

```
1 ArrayList<Integer> scores = new ArrayList<>();  
2 scores.add(90);           // autobox int -> Integer  
3 scores.add(85);  
4 int s = scores.get(0);    // auto-unbox
```

You cannot write `ArrayList<int>` — use `Integer`, `Double`, `Boolean`, ...

Looping

```
1 for (int i = 0; i < names.size(); i++) {  
2     System.out.println(i + ": " + names.get(i));  
3 }  
4  
5 for (String n : names) {  
6     System.out.println(n);  
7 }
```

Prefer `size()` (method) — not a `.length` field.

Quiz 2 — What prints?

```
1 ArrayList<Integer> xs = new ArrayList<>();  
2 xs.add(10);  
3 xs.add(20);  
4 xs.add(30);  
5 xs.remove(1);  
6 System.out.println(xs.get(1));
```

Quiz

Output?

1. 10
2. 20
3. 30
4. Error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. 30 — removing index 1 drops 20; former index 2 shifts to 1.

Reminder: a simple model class

```
1 public class Student {  
2     private String name;  
3     private int id;  
4  
5     public Student(String name, int id) {  
6         this.name = name;  
7         this.id = id;  
8     }  
9  
10    public String getName() { return name; }  
11    public int getId() { return id; }  
12  
13    public String summary() {  
14        return name + " (" + id + ")";  
15    }  
16 }
```

Roster as ArrayList

```
1 ArrayList<Student> roster = new ArrayList<>();  
2 roster.add(new Student("Ada", 1001));  
3 roster.add(new Student("Alan", 1002));  
4  
5 for (Student s : roster) {  
6     System.out.println(s.summary());  
7 }
```

```
1 public static Student findById(ArrayList<Student> list, int id) {  
2     for (Student s : list) {  
3         if (s.getId() == id) {  
4             return s;  
5         }  
6     }  
7     return null; // not found -- caller must check  
8 }
```

Common mistake

Using the result of `findById` without checking for `null`.

Case study: CourseRoster

Design a small class that:

- Stores students in a private `ArrayList`
- Can add a student
- Can print all summaries
- Can count how many students are enrolled

CourseRoster (sketch)

```
1 import java.util.ArrayList;
2
3 public class CourseRoster {
4     private ArrayList<Student> students = new ArrayList<>();
5
6     public void add(Student s) {
7         students.add(s);
8     }
9
10    public int size() {
11        return students.size();
12    }
13
14    public void printAll() {
15        for (Student s : students) {
16            System.out.println(s.summary());
17        }
18    }
19 }
```

Demo main

```
1 public class Main {  
2     public static void main(String[] args) {  
3         CourseRoster cs101 = new CourseRoster();  
4         cs101.add(new Student("Ada", 1001));  
5         cs101.add(new Student("Grace", 1003));  
6         System.out.println("Enrolled: " + cs101.size());  
7         cs101.printAll();  
8     }  
9 }
```

What this practice combines

- Classes + constructors + encapsulation
- ArrayList as internal storage
- Clear public operations (`add`, `printAll`, `size`)

This is the everyday shape of many beginner OOP programs.

Quiz

In CourseRoster, why keep the ArrayList private?

1. So nobody outside can break roster rules by editing the list directly
2. Because ArrayList cannot be public in Java
3. So main runs faster
4. Because private lists cannot grow

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 3

Answer

1. Encapsulation: clients use `add/printAll`, not raw list edits.

Quiz

True or false?

`ArrayList<Student>` can grow as you enrol more students without choosing a fixed capacity up front.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

True. That growable behaviour is a main reason to use ArrayList.

Pitfalls

- Using `.length` on an `ArrayList` — use `size()`
- Forgetting `import java.util.ArrayList;`
- Removing elements while using a simple index loop carelessly
- Confusing index with object identity

Practice before next time

Try this

Extend `CourseRoster` with `findById` and `containsName(String)`. Write short tests in `main`.

Stretch: remove a student by id (return whether it succeeded).

Where the course sits now

You can:

1. Use core syntax (Lecture 02)
2. Structure behaviour with methods (03)
3. Model things with classes/objects (04)
4. Protect state and use `static` wisely (05)
5. Manage growing collections of objects (06)

Key takeaways

1. ArrayList: growable list of objects (wrappers for primitives)
2. Core API: add, get, remove, size, for-each
3. Lists of your classes enable realistic programs
4. Keep collections private inside a managing class when it helps

ArrayList API or the roster design—what should we rework live?